

CPP3-20 Design Document

Fault Tolerant, Distributed in-memory KV Store

Team: CPP3-20

Project: Fault Tolerant, Distributed in-memory KV Store

Component: Consistent Hashing with Virtual Nodes, Buddy Replication, and Membership Service

Status: Pending Approval Before Implementation

Table of Contents

1. [Abstract](#)
 2. [Problem Statement and Limitations of the Current Design](#)
 3. [Goals and Non-Goals](#)
 4. [Basic Concepts](#)
 5. [Proposed Architecture](#)
 6. [Data Structures](#)
 7. [Key Distribution and Lookup Algorithm](#)
 8. [Virtual Nodes](#)
 9. [Buddy Replication](#)
 10. [Membership Service](#)
 11. [Node Addition and Removal](#)
 12. [Primary Revival and Hinted Handoff](#)
 13. [Proposed Integration Points with Existing Codebase](#)
 14. [Proposed Interface Changes](#)
 15. [Failure Scenarios and Handling](#)
 16. [Industry Case Studies](#)
 17. [Future Scope / Improvements](#)
-

1. Abstract

The current system distributes keys across nodes using a simple `key % number_of_nodes` modulo hash. This approach is fast but has a fundamental architectural flaw: when a node is

added or removed, almost every key remaps to a different node. In a cluster of N nodes, removing one node causes approximately $(N-1)/N$ of all keys to move, which is a near-total redistribution event that is unacceptable in a production HPC environment.

This project proposes replacing the modulo distributor with a **consistent hash ring**. Under consistent hashing, adding or removing a single node causes only approximately $1/N$ of all keys to move, regardless of cluster size. The design additionally incorporates:

- **Virtual nodes** to ensure balanced load distribution across physical nodes with unequal token counts on the ring.
- **Buddy node replication** to provide fault tolerance: each key is held by a primary node and replicated to one successor (buddy) node on the ring, so that a single node failure does not cause data loss.
- A **Membership Service** that uses gossip to maintain the live view of cluster nodes, detect joins and failures, and propagate ring topology changes to all clients, with a coordinator-based fallback path if the gossip implementation cannot be completed on the initial deadline.
- **Node addition and removal protocols** that move only the minimal set of affected keys when the cluster topology changes.
- **Hinted handoff** as an optional enhancement if time permits, to ensure writes destined for a temporarily-unavailable primary are not dropped and are delivered once the primary recovers.

This document also includes an industry case study section examining how Redis Cluster, Apache Cassandra, and Amazon DynamoDB solve the same distributed routing and fault tolerance problems.

2. Problem Statement and Limitations of the Current Design

The existing key distribution logic lives in `KVDistributor` (`src/KVDistributor.cpp` , `include/KVDistributor.hpp`) and uses:

```
int KVDistributor::getNodeId(int key) {  
    return key % count_of_node;  
}
```

2.1 Problems with Modulo Hashing

Massive redistribution on topology change. If the cluster has 2 nodes and a third is added, approximately $2/3$ of all existing keys hash to a different node than before. There is currently no

mechanism to move those keys; they simply become unavailable or stale on the wrong node. The project requirement calls for dynamic node addition and removal, which makes this a blocker.

No fault tolerance. Each key lives on exactly one node. If that node crashes, its data is gone until the node is restored. There is no replication.

Load imbalance with heterogeneous nodes. Modulo hashing implicitly assigns each node an equal share, which is only correct if all nodes have equal capacity.

No dynamic membership awareness. The current design reads node endpoints from a static `config.json` at startup. If a node goes down or a new node is added at runtime, no client learns of this change automatically. The ring once implemented must be backed by a live membership service to remain consistent with actual cluster state.

No recovery path for failed primaries. If the primary for a key goes down and later restarts, writes accepted by the buddy during the outage are not automatically transferred back. Without a hinted handoff mechanism, or if that feature is deferred to a later phase, the revived primary starts with stale or missing data.

2.2 What Consistent Hashing Solves

Consistent hashing maps both nodes and keys onto a fixed circular address space (the "ring"). Adding or removing a node affects only keys that land between the affected node's position and its nearest neighbor on the ring. All other keys remain on the same node. With virtual nodes, each physical node occupies multiple positions on the ring, smoothing out load imbalance.

3. Goals and Non-Goals

3.1 Goals

- Replace `KVDistributor::getNodeId()` and associated logic with a consistent hash ring implementation.
- Each physical node is represented by a configurable number of virtual nodes (tokens) on the ring.
- Each key is owned by exactly one **primary** physical node.
- Each key is additionally replicated to one **buddy** physical node (the next distinct physical node clockwise on the ring from the primary).
- A **Membership Service** component uses gossip to track live nodes, propagates membership state to all `KVDistributor` instances, and keeps a coordinator fallback available for a short-deadline implementation path.

- **Node addition** uses lazy migration: the new node begins serving writes immediately after joining; keys that now belong to it are migrated from their previous owner on demand (read-triggered) or via a background scan, with no upfront bulk transfer required. Node removal triggers transfer of the departing node's keys to its successor before shutdown.
- **Hinted handoff** is optional for this phase and will be implemented if time permits.
- The `KVDistributor` public interface (`get`, `insert`, `update`, `deleteKey`) must continue to work identically from the perspective of `main_client.cpp`.
- The design must be compatible with the existing Thallium RPC layer (`KVClient`, `KVServer`) with no changes to the wire protocol.
- The design must be compatible with both `StorageMode::MEMORY` and `StorageMode::PERSISTENT`.

3.2 Non-Goals

- Replication to more than one buddy per key.
- Consensus or leader election between replicas.
- Automatic re-replication after a node failure beyond what hinted handoff provides.

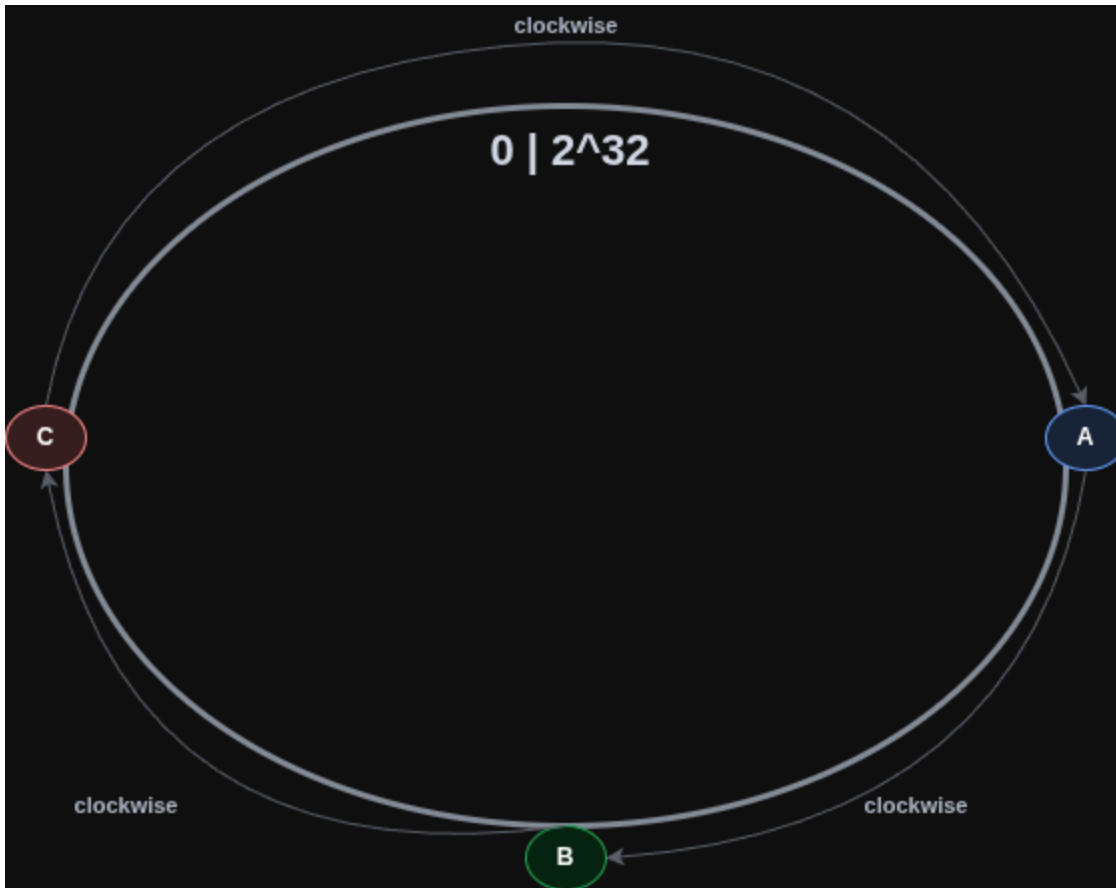
4. Basic Concepts

4.1 The Hash Ring

A consistent hash ring is a circular integer space, conventionally `[0, 232 - 1]` when using a 32-bit hash function, or `[0, 264 - 1]` for 64-bit. Both nodes and keys are mapped onto this ring by hashing their identifiers.

Ring space: 0 ————— 2³² - 1
(wraps around)

Example with 3 nodes (A, B, C), no virtual nodes:



Keys are assigned to the first node clockwise from their hash position.

4.2 Key Assignment Rule

Given a key k :

1. Compute $h = \text{hash}(k)$, placing k at position h on the ring.
2. Walk clockwise from h until the first node token is found.
3. The physical node owning that token is the **primary** for key k .

4.3 Why 32-bit Ring

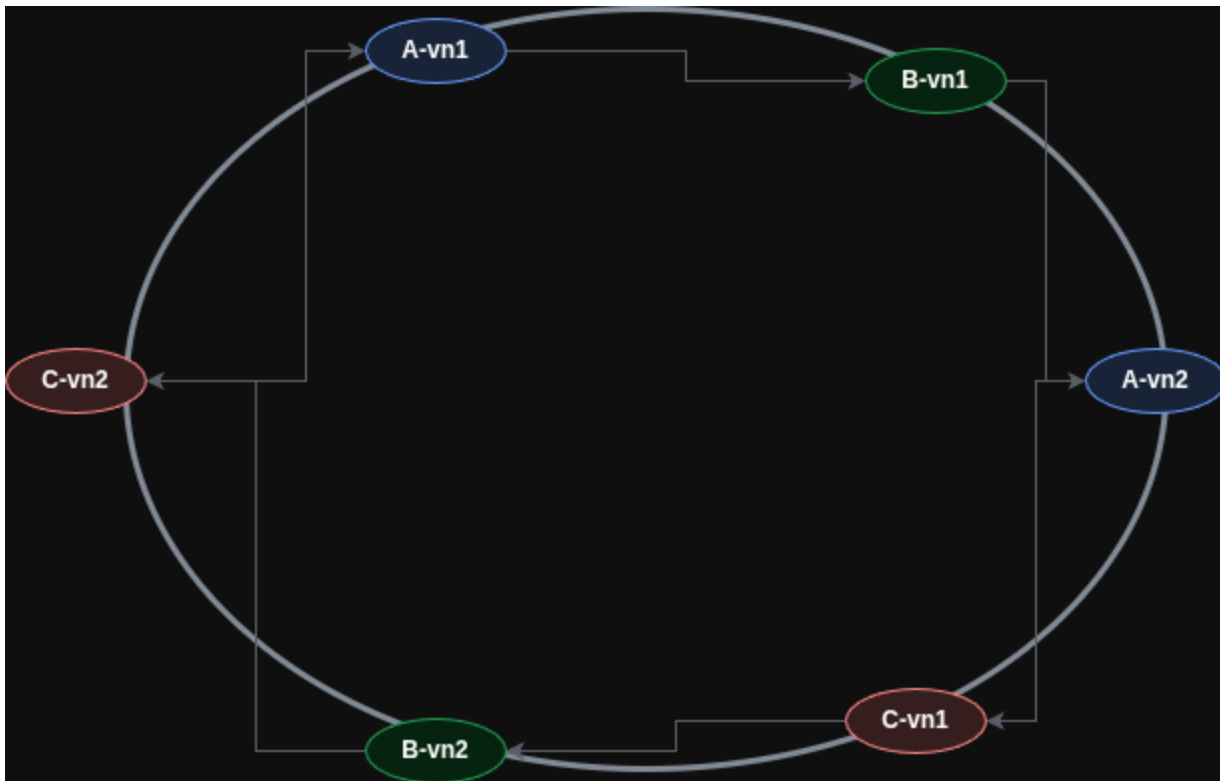
The current system uses integer keys. Since keys are `int` (32-bit signed), a 32-bit ring is sufficient. MurmurHash3 (32-bit output variant) is appropriate: it is non-cryptographic, extremely fast, has good distribution, and has no external library dependency (single header/source file).

4.4 Virtual Nodes (Tokens)

With only one ring position per physical node, random placement can cause severe load imbalance; one node might own 60% of the ring and another only 5%. Virtual nodes solve this: each physical node is hashed multiple times to produce V positions on the ring. With sufficiently large V (typically 100–200 per node), the load across nodes converges toward $1/N$ each regardless of the number of physical nodes.

4.5 Buddy Replication

The buddy of a key's primary node is the next **distinct physical node** clockwise on the ring from the primary. "Distinct physical node" is important: it must skip over virtual nodes of the same physical node.



Key k lands between B-vn1 and A-vn2.

Primary = A (owner of A-vn2, the first token clockwise from k)

Buddy = C (the first *different physical node* clockwise from A-vn2)

On writes (insert, update, delete), the `KVDistributor` sends the operation to both primary and buddy. On reads (get), it queries the primary and falls back to the buddy if the primary is unreachable.

4.6 Membership Service

A membership service maintains the authoritative, live view of cluster membership. Without it, the consistent hash ring is a static data structure that goes stale the moment any node joins or leaves. The preferred design uses gossip to spread membership updates and failure suspicion across the cluster, with a coordinator-based fallback available if the gossip path is not ready in time. With a membership service:

- Nodes announce themselves on startup and then participate in periodic gossip exchanges.
- Gossip lets the service detect node joins, departures, and silent failures (crashes, network partitions).

- Ring topology changes are propagated to all clients, which rebuild their local `ConsistentHashRing` from the updated membership list.

We understood that this is the standard pattern which is used in production systems: Cassandra uses gossip-based membership, Redis Cluster uses a cluster bus with PING/PONG messages, and DynamoDB uses a centralized membership management plane. The design below uses gossip as the primary membership mechanism, while keeping a lightweight coordinator-based fallback path appropriate for the scale of this project.

4.7 Hinted Handoff (Optional)

When a primary node is temporarily unavailable, writes intended for it must not be silently dropped. Hinted handoff is the mechanism by which the buddy (or another designated node) accepts these writes, stores them tagged with the intended destination, and replays them to the primary once it comes back online. This is the same technique used by Amazon DynamoDB and Apache Cassandra to provide high write availability without sacrificing durability. It is optional for this phase and will be implemented if time permits.

5. Proposed Architecture

5.1 Component Overview

Component	Status	Responsibility
<code>ConsistentHashRing</code>	New	Ring construction, primary/buddy lookup
<code>MembershipService</code>	New	Gossip-based membership, topology broadcast, coordinator fallback
<code>MembershipClient</code>	New	Client-side membership subscription and ring rebuild
<code>HintStore</code>	Optional	Temporary storage of hinted writes pending handoff
<code>KVDistributor</code>	Modified	Uses ring + membership instead of modulo; optionally issues hinted writes
<code>Config</code>	Modified	Adds <code>virtual_nodes_per_node</code> field
<code>KVServer</code> / <code>KVClient</code> / <code>KvStore</code>	Unchanged	No interface or implementation changes

5.2 Proposed New Files

- `include/ConsistentHashRing.hpp / src/ConsistentHashRing.cpp`
- `include/MembershipService.hpp / src/MembershipService.cpp`
- `include/MembershipClient.hpp / src/MembershipClient.cpp`
- `include/HintStore.hpp / src/HintStore.cpp`
- `src/MurmurHash3.cpp / include/MurmurHash3.hpp`

5.3 Unchanged Components (susceptible to change)

- `KvStore` (storage layer)
 - `KVServer` (Thallium provider, RPC handlers)
 - `KVClient` (Thallium client, RPC callers)
 - `main_server.cpp`
 - `main_client.cpp`
-

6. Data Structures

6.1 Token

A token represents one virtual node position on the ring.

```
struct Token {
    uint32_t position; // Position on the ring [0, 2^32 - 1]
    int      node_id;  // Physical node this token belongs to
};
```

6.2 Ring Storage

The ring is stored as a `std::vector<Token>` sorted in ascending order of `position`. Lookup uses `std::lower_bound` (binary search, $O(\log N \cdot V)$ where N = node count, V = virtual nodes per node).

```
std::vector<Token> ring_; // sorted by Token::position
```

This is preferred over `std::map<uint32_t, int>` due to lower per-element overhead for read-heavy workloads. Insertions happen only at startup or on membership events, so the cost of re-sorting on topology change is amortized.

6.3 Node Metadata


```
std::unordered_map<int, std::string> node_endpoints_;  
// node_id → "ofi+tcp://ip:port"
```

6.4 Membership Record

The membership service maintains the following per-node record:

```
struct MemberRecord {  
    int      node_id;  
    std::string endpoint;  
    NodeStatus status;           // ALIVE, SUSPECT, DEAD  
    uint64_t last_heartbeat_ms;  // epoch milliseconds  
    uint32_t generation;         // incremented on each restart  
};
```

The `generation` counter is critical for distinguishing a restarted node from a stale heartbeat — a node that crashes and restarts increments its generation, allowing the membership service and hinted handoff logic to correctly identify it as a fresh join rather than a recovery of the previous instance.

6.5 Hint Record

```
struct HintRecord {  
    int      intended_node_id;    // the node this write was meant for  
    int      key;  
    std::string value;  
    HintOp   op;                 // INSERT, UPDATE, DELETE  
    uint64_t timestamp_ms;  
    uint32_t target_generation;   // generation of the node at time of  
    hint  
};
```

Hints are persisted to disk (a simple append-only log under `hints/<node_id>/`) so that the holder of the hints survives crashes before delivery is complete.

7. Key Distribution and Lookup Algorithm

7.1 Hashing a Key to Ring Position

Keys are `int`. They are hashed to `uint32_t` ring positions using MurmurHash3 (32-bit variant):

```
ring_position(key) = murmur3_32(&key, sizeof(int), seed=0)
```

MurmurHash3 has excellent avalanche properties for small integer inputs, avoiding clustering that would occur with naive `(uint32_t)key` casting.

7.2 Hashing a Node Token to Ring Position

```
for v in 0..V-1:
    token_str = "node_" + str(node_id) + "_vn_" + str(v)
    position = murmur3_32(token_str.data(), token_str.size(), seed=0)
    ring_.push_back(Token{position, node_id})
```

After all tokens are generated, `ring_` is sorted by `position`. Collision tiebreaking: lower `node_id` wins, resolved by `std::stable_sort` with comparator `(a.position < b.position) || (a.position == b.position && a.node_id < b.node_id)`.

7.3 Primary Node Lookup

```
getPrimaryNode(key):
    pos = ring_position(key)
    it = lower_bound(ring_, pos)
    if it == ring_.end(): it = ring_.begin() // wrap around
    return it->node_id
```

Cost: $O(\log N \cdot V)$.

7.4 Buddy Node Lookup

```
getBuddyNode(key):
    pos = ring_position(key)
    primary_id = getPrimaryNode(key)
    it = lower_bound(ring_, pos)
    if it == ring_.end(): it = ring_.begin()
    start = it
    loop:
        advance it (wrapping at end)
        if it->node_id != primary_id: return it->node_id
        if it == start: return primary_id // single-node cluster
```

8. Virtual Nodes

8.1 Why Virtual Nodes Are Needed

With a small number of physical nodes and one token per node, arc lengths (fraction of the ring owned by each node) are highly variable. For 2 nodes with one token each, one could own a big majority of the ring, which will result in very bad load handling.

With some calculations, we understood that at a value of $V = 150$ virtual nodes per physical node, the standard deviation of load fraction per node is approximately $1/\sqrt{V} \approx 8\%$ of the mean. Memory cost: at $V = 150$ and $N = 100$ nodes, the ring holds 15,000 tokens at 8 bytes each = 120 KB, which is negligible. This seemed to be a good default value for a ring size of 2^{32} .

8.2 Configurable V

`virtual_nodes_per_physical_` is a config parameter with a default of 150. The config file gains an optional field:

```
"virtual_nodes_per_node": 150
```

If absent, 150 is used. For the current 2-node cluster this produces 300 ring tokens, which is more than sufficient for balanced distribution.

9. Buddy Replication

9.1 Write Path

On any mutating operation (`insert` , `update` , `deleteKey`), `KVDistributor` will:

1. Determine `primary_id = ring.getPrimaryNode(key)` .
2. Determine `buddy_id = ring.getBuddyNode(key)` .
3. Execute the operation on the primary (local store if `primary_id == local_node_id` , else RPC via `KVClient`).
4. Execute the same operation on the buddy (local store if `buddy_id == local_node_id` , else RPC via `KVClient`).

Both operations are performed **synchronously** in this phase. The operation is considered complete from the client's perspective as soon as step 3 succeeds. Buddy replication is best-effort.

9.2 Read Path

On `get(key)` :

1. Determine `primary_id`.
2. Attempt to read from primary (local or RPC).
3. If primary read fails (RPC exception or `key_not_found`), fall back to buddy. There can be two cases for this failure.
 - The primary is a new node, in which case the node will sync the KV pair with the buddy node (lazy migration).
 - The primary is a previously failed node, in which case it will obtain the hints from its buddy nodes (if hinted handoffs mechanism is implemented).
4. If buddy read also fails, return the error string.

9.3 Self-Write Deduplication

If `primary_id == buddy_id` (only possible with exactly one physical node), no duplicate write is issued.

9.4 Buddy Write Failure Behavior

If the buddy write fails:

- The primary write has already succeeded.
- Logged: `[WARN] Buddy write failed for key <k> on node <buddy_id>: <reason>`.
- Not retried automatically.
- The key remains on the primary only until the buddy recovers.

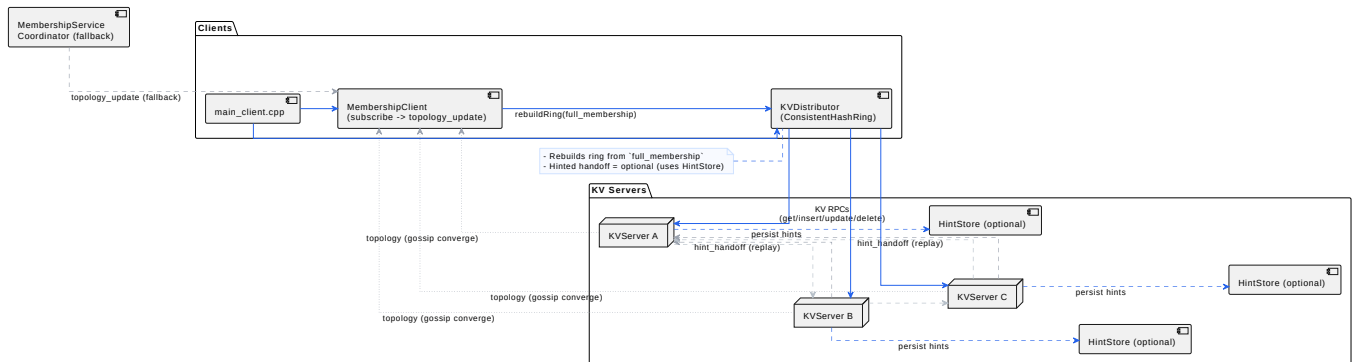
10. Membership Service

10.1 Overview

The membership service is a dedicated process (or a designated node role) that maintains the authoritative live membership list. The primary plan is to use gossip so nodes can exchange membership state, announce joins, and detect nodes going away without a central dispatcher. All `KVdistributor` instances subscribe to the resulting updates and rebuild their local `ConsistentHashRing` whenever the membership changes.

Without a membership service, each `kvm_client` constructs its ring independently from `config.json` and has no way to learn of node failures or new joins at runtime. This is correct only for a fully static cluster. The membership service is what transforms the consistent hash ring from a static routing table into a dynamic, self-updating one. If the gossip implementation is cut for time, the coordinator-based fallback keeps the system usable for this phase.

10.2 Architecture

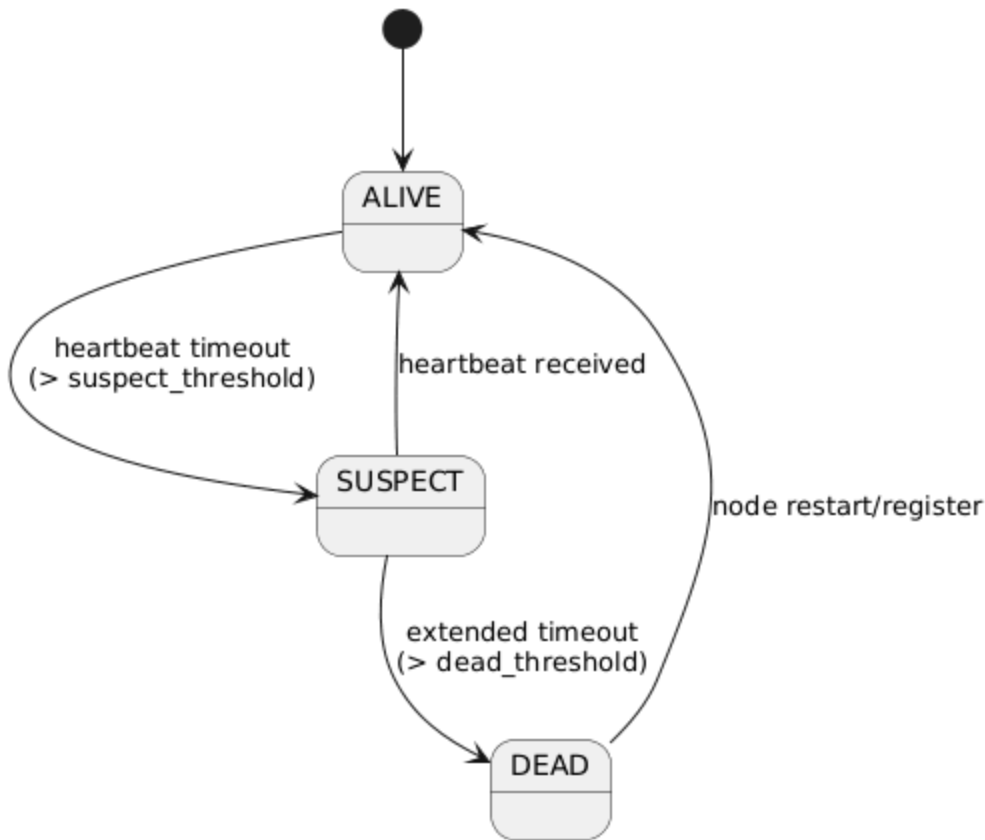


Note: This architecture assumes that there's only 3 nodes so far in the ring and hence, the new node

10.3 Gossip and Failure Detection

- Each `kvm_server` instance gossips its own membership state to peers at a fixed interval.
- Gossip payloads include `node_id`, `endpoint`, `generation`, and the latest local view of peer status.
- The cluster marks a node as `SUSPECT` when gossip stops arriving or peers stop forwarding fresh state for that node within `suspect_threshold_ms` (default: 3000 ms).
- A `SUSPECT` node is marked `DEAD` if the gossip view does not recover within `dead_threshold_ms` (default: 10000 ms).
- A `DEAD` node triggers a topology change event.

If the gossip path is not ready in time, the coordinator fallback can still accept explicit node registration and failure notifications using the same membership records.



Some default values for the thresholds are:

```

heartbeat interval:  1000 ms    (configurable)
suspect threshold:   3000 ms    (3x heartbeat interval)
dead threshold:      10000 ms   (10x heartbeat interval)
  
```

10.4 Node Registration and Deregistration

On startup: Each `kvm_server` announces itself to the gossip layer with `MembershipService::register_node(node_id, endpoint, generation)`. The service adds the node to its membership list with status `ALIVE` and broadcasts a `NODE_JOINED` event to all subscribed clients.

On clean shutdown: Each `kvm_server` calls `MembershipService::deregister_node(node_id)`. The service removes it and broadcasts `NODE_LEFT`.

On crash (no deregister): The gossip failure detector notices the missing node, transitions it through `SUSPECT` → `DEAD`, then broadcasts `NODE_FAILED`.

10.5 Topology Change Propagation

When the membership list changes (join, leave, or failure), the `MembershipService` pushes a `TopologyUpdate` to all registered `MembershipClient` instances via RPC. In the gossip-first design, this update is a convergence signal after the cluster has exchanged membership state; in the fallback mode, it is sent directly by the coordinator:

```
struct TopologyUpdate {
    ChangeType          change;          // NODE_JOINED, NODE_LEFT,
    NODE_FAILED
    int                 node_id;
    std::string          endpoint;
    uint32_t             generation;
    std::vector<MemberRecord> full_membership; // current live set
};
```

Each `MembershipClient` receives the update and calls `KVDistributor::rebuildRing(full_membership)`, which reconstructs the `ConsistentHashRing` from the new live node set.

10.6 MembershipService RPC Interface

```
// Exposed by MembershipService via Thallium
void register_node(int node_id, std::string endpoint, uint32_t generation);
void deregister_node(int node_id);
void heartbeat(int node_id, uint32_t generation);

// Exposed by MembershipClient via Thallium (called by MembershipService)
void topology_update(TopologyUpdate update);
```

10.7 Membership Service Placement

The preferred membership path is gossip between nodes, but the implementation keeps a designated coordinator node available as a fallback path for a short-deadline build. If the gossip workflow slips, the coordinator can still anchor registration and topology propagation without blocking the rest of the system. A highly available coordinator would be a future work item.

10.8 Config Changes for Membership Service

```
"membership_coordinator": "ofi+tcp://192.168.1.10:7000",
"heartbeat_interval_ms": 1000,
"suspect_threshold_ms": 3000,
"dead_threshold_ms": 10000
```

11. Node Addition and Removal

11.1 Overview

This section defines the protocol for node addition and removal. Both operations are gated through the `MembershipService`: a node cannot join or leave in a way that bypasses the membership view, as doing so would create split-brain scenarios where different clients have different views of the ring. Gossip is the preferred way to propagate those membership changes, while the coordinator remains a fallback if the gossip flow is not ready yet.

Node **addition** uses **lazy migration**: the new node is inserted into the ring immediately and begins accepting writes for its new key ranges right away. Keys that it now owns but has not yet received are migrated on demand: either triggered by a read miss or via a background scan, rather than by an upfront bulk transfer. This avoids blocking the join on a potentially large data movement.

Node **removal** (clean shutdown) requires an eager transfer: the departing node pushes its keys to its successor before deregistering, since once it leaves the ring there is no fallback path for those keys (unlike addition, where the old owner is still live and can serve reads during the migration window).

11.2 Node Addition Protocol (Lazy Migration)

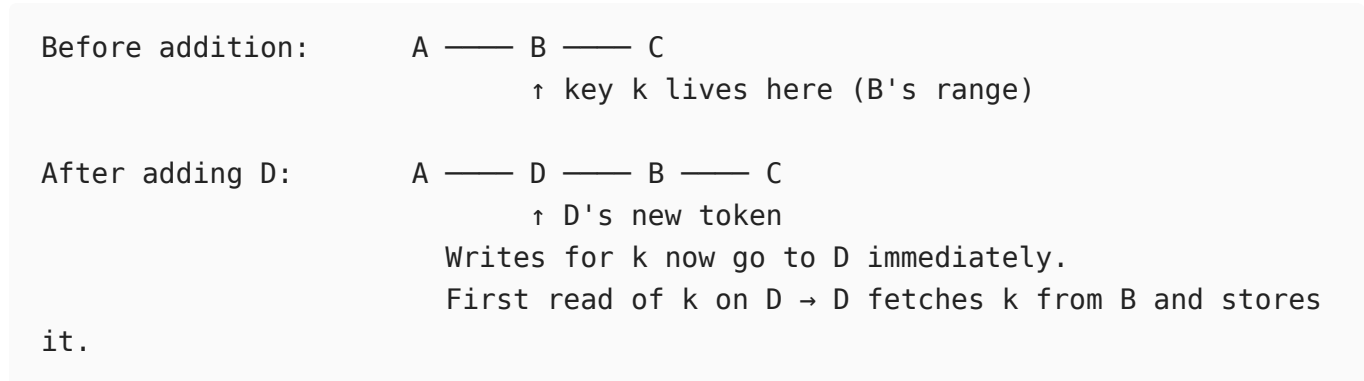
When a new node `N_new` joins the cluster:

1. `N_new` starts its `kvm_server`, initializes `HintStore` (optional), and begins the membership join process. Preferred path: `N_new` joins the gossip mesh by contacting a set of seed peers and advertising `{node_id, endpoint, generation, status=ALIVE}` in its gossip payloads. Fallback path: if gossip is not yet available, `N_new` may call `MembershipService::register_node(node_id, endpoint, generation)` on the configured coordinator to register explicitly.
2. Once the membership change is accepted (either after gossip convergence or when the coordinator records the registration), the membership system broadcasts `NODE_JOINED(N_new)` to all subscribed clients. In the gossip-first flow this broadcast is a convergence signal emitted after peers have merged the new record; in the fallback flow the coordinator issues the `NODE_JOINED` update directly.
3. Each client rebuilds its `ConsistentHashRing` with `N_new` inserted. From this point, the ring routes writes for `N_new`'s ranges directly to `N_new`.
4. `N_new` starts empty. No upfront bulk transfer occurs to ensure availability of all nodes.

Read miss path (on-demand migration): When a client reads a key `k` whose ring primary is now `N_new` but `N_new` does not have it yet, `N_new` fetches the key from the previous owner

(the node that was primary for `k` before `N_new` joined), stores it locally, and returns it. From that point, `N_new` is authoritative for `k`.

Background scan (optional): In parallel, `N_new` may iterate over all keys it should own by scanning the ring token boundaries and issuing bulk fetch RPCs to previous owners. This fills the cache proactively and reduces read-miss latency for cold keys.



Consistency during migration window: Writes always go to the current ring primary (`N_new`). Reads that miss on `N_new` fall back to the previous owner transparently. Once a key has been migrated (either on-demand or via background scan), the previous owner's copy becomes stale and may be lazily deleted.

11.3 Node Removal Protocol (Clean Shutdown)

When a node `N_rem` is gracefully removed:

1. `N_rem` calls `MembershipService::deregister_node()`.
2. The `MembershipService` broadcasts `NODE_LEFT(N_rem)`.
3. Each client rebuilds its ring without `N_rem`.
4. The keys formerly owned by `N_rem` as primary now belong to the successor node `N_succ` (the next distinct physical node clockwise). Since `N_rem`'s buddy has a copy of each such key, no data is lost.
5. `N_rem` transfers its keys to `N_succ` (the new primary) and ensures the buddy for each migrated key is updated.
6. `N_rem` shuts down cleanly.

11.4 Node Removal: Failure (Unclean)

If `N_rem` fails without a clean shutdown:

1. The `MembershipService` detects the absence via heartbeat timeout and broadcasts `NODE_FAILED(N_rem)`.
2. Clients rebuild the ring without `N_rem`.

3. The buddy of each key formerly owned by `N_rem` becomes the de facto primary until `N_rem` recovers or re-replication is triggered.
 4. New writes for those keys go to the new primary (the old buddy), and a new buddy is selected (the next distinct physical node from the new primary).
 5. The old buddy now needs a new buddy for those keys; this re-replication step is handled as a background repair triggered by the `NODE_FAILED` event.
-

12. Primary Revival and Hinted Handoff

12.1 The Problem

When a primary node `P` goes down, writes continue to be accepted by the buddy `B` (which becomes the acting primary). When `P` comes back online, it has missed all writes that occurred during its absence. Without a recovery mechanism, `P` holds stale data and cannot safely serve reads again until it is fully synchronized with `B`.

Hinted handoff solves this by having the buddy durably store all writes it accepted "on behalf of" `P` during the outage, tagged with `P`'s identity and generation, and replaying them to `P` once it reconnects. This is an optional enhancement for this phase and can be deferred if time is tight.

12.2 Hinted Write Path

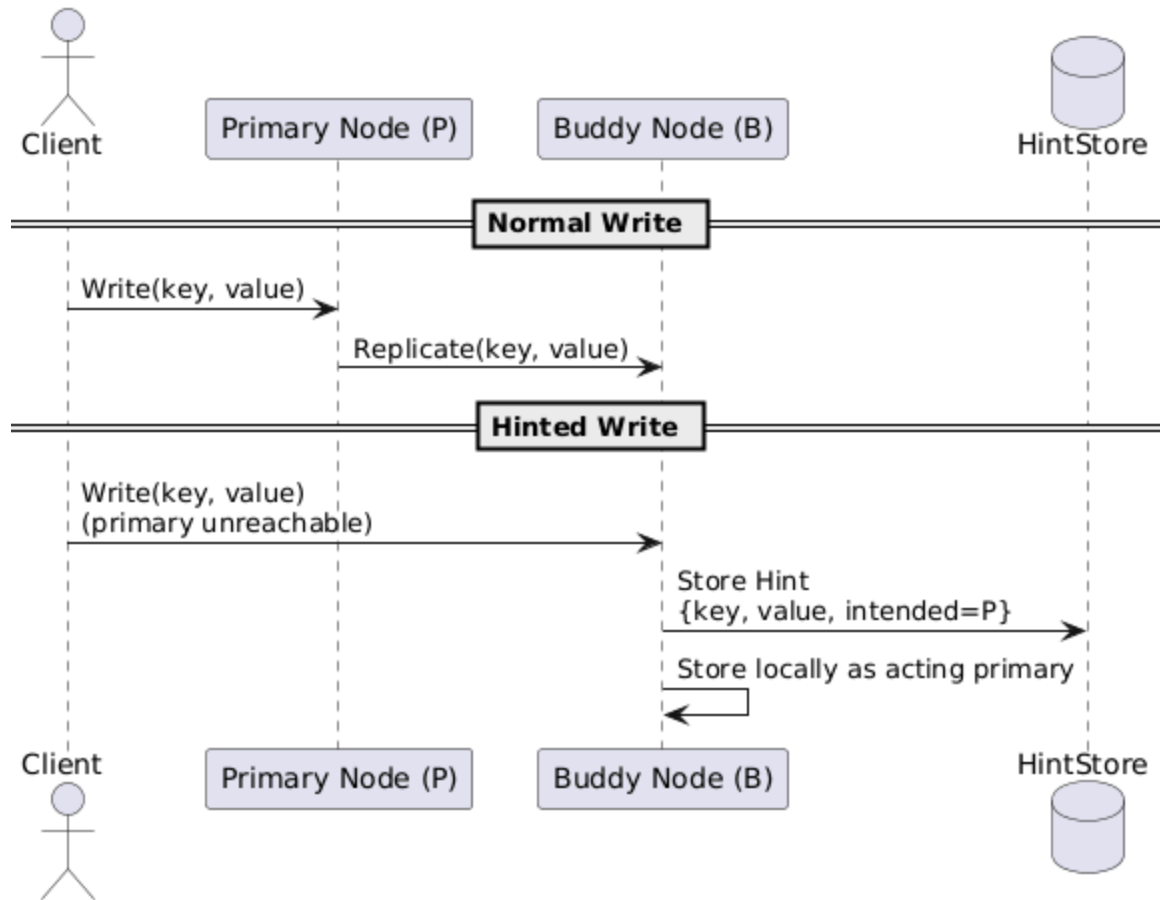
When `KVDistributor` determines that the primary `P` is unreachable:

1. The write is sent to the buddy `B` as a normal write (the buddy stores it in its `KvStore`).
2. Additionally, `B` stores a `HintRecord` in its `HintStore` for the intended primary `P`:

```
HintRecord {
    intended_node_id = P.node_id,
    key              = k,
    value            = v,
    op               = INSERT | UPDATE | DELETE,
    timestamp_ms     = now(),
    target_generation = P.last_known_generation
}
```

3. The `HintStore` is persisted to disk (append-only log under `hints/<node_id>/`) so hints survive crashes of `B` before delivery.

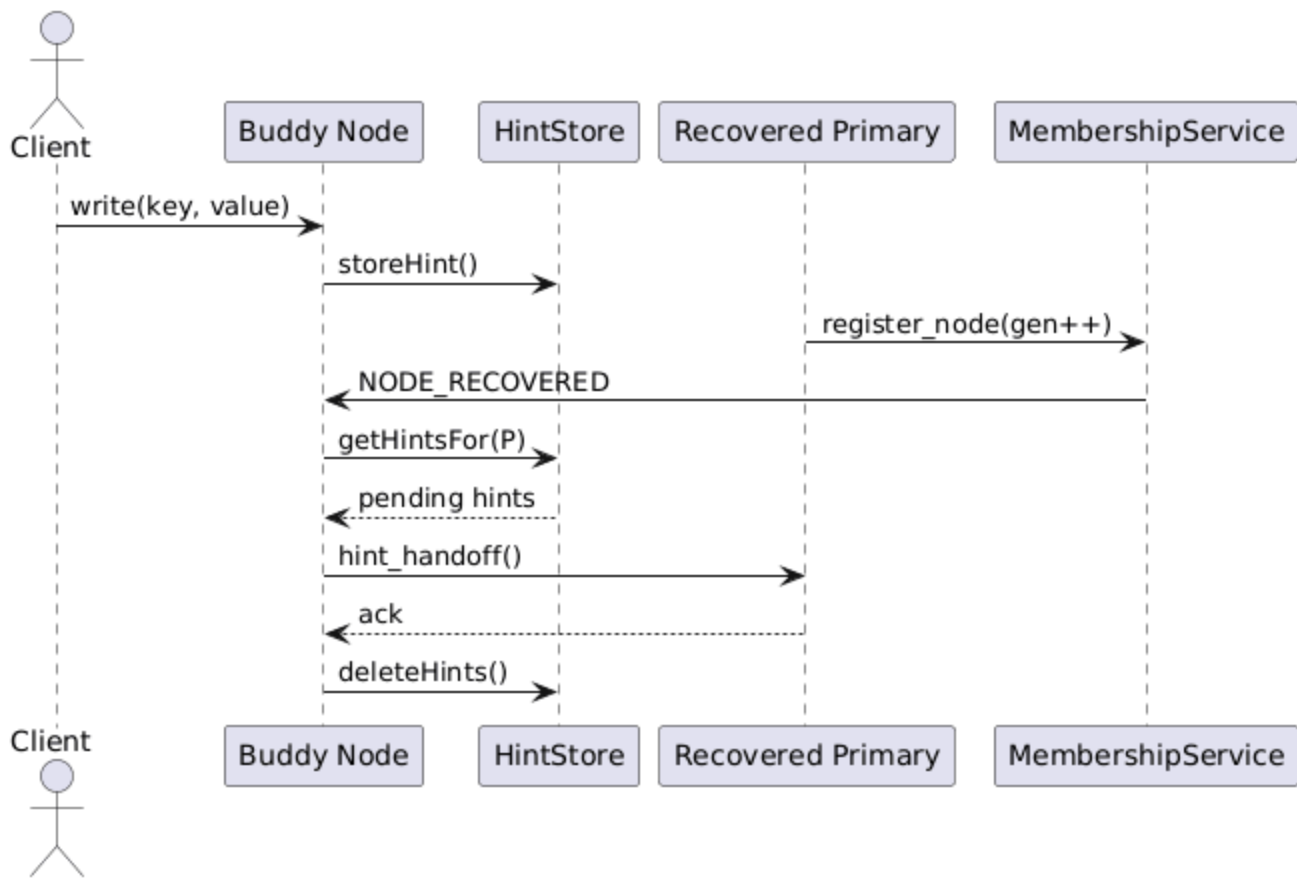
4. From the client's perspective, the write succeeds (the data is safely stored on B).



12.3 Primary Recovery and Hint Delivery

When P comes back online:

1. P registers with the MembershipService with an incremented generation.
2. The MembershipService broadcasts `NODE_RECOVERED(P, new_generation)`.
3. B receives this event and checks its HintStore for any records with `intended_node_id == P.node_id` and `target_generation == P.last_generation` (i.e., hints accumulated during P's absence, not from a prior incarnation of P).
4. B replays each hint to P via a `hint_handoff` RPC, in timestamp order.
5. P applies each hinted write to its KvStore.
6. Once all hints are successfully delivered, B deletes the corresponding hint records.
7. P is now fully synchronized and can resume serving as primary.



12.4 Generation Mismatch Handling (Optional)

The `generation` field handles the case where `P` crashes, hints accumulate on `B`, and then `P` restarts, but before all hints are delivered, `P` crashes again. On the second restart, `P`'s generation increments again. `B` must only replay hints tagged with `target_generation = P.generation - 1` (the previous session). Hints from even older sessions can be discarded if `P` is considered to have started fresh.

In short: `B` delivers hints where `target_generation == P.current_generation - 1`. Older hints are stale and deleted without delivery.

12.5 Hint Expiry

Hints that are not delivered within `hint_expiry_ms` (configurable, default: 1 hour) are dropped with a warning log. This prevents unbounded growth of the hint store in scenarios where `P` is down for an extended period and eventually replaced by a new node. When hints expire, the key range affected must be treated as potentially inconsistent and should trigger a background reconciliation (out of scope right now, but noted as a future work item).

12.6 HintStore Interface

```

class HintStore {
public:

```

```

HintStore(const std::string& storage_path, int local_node_id);

// Called on write to an unreachable primary
void storeHint(const HintRecord& hint);

// Called when a target node is detected as recovered
std::vector<HintRecord> getHintsFor(int target_node_id, uint32_t
target_generation);

// Called after successful delivery
void deleteHints(int target_node_id, uint32_t target_generation);

// Called on startup to recover pending hints after a crash
std::vector<HintRecord> loadAllHints();

// Called periodically to remove expired hints
void expireHints(uint64_t expiry_threshold_ms);
};

```

12.7 Interaction Between Hinted Handoff and Buddy Replication

It is important to distinguish the two replication paths:

Scenario	Mechanism	Who holds the data
Normal operation	Buddy replication	Primary and buddy both hold the key
Buddy is down	Primary write only; buddy syncs on recovery	Primary only during outage
Primary is down	Hinted write to buddy	Buddy holds data + hint for primary
Both down	Write fails	No data durability guarantee

Hinted handoff fills the gap in the "primary is down" case and it ensures that when the primary returns, it receives all writes it missed, rather than serving stale reads.

13. Proposed Integration Points with Existing Codebase

Note: The code snippets below are very minimal and may not be implemented, as it is, in the final implementation. It's here to provide a better clarity and visual on the changes being made on the existing codebase.

13.1 KV Distributor Constructor

Current:

```
KVDistributor::KVDistributor(KvStore& kv_store, const Config& config)
    : kv(kv_store), config(config),
      count_of_node(config.read_count()),
      protocol(config.read_protocol()),
      provider_id(1),
      kv_client(protocol, 1) {
    for (int i = 0; i < count_of_node; ++i) {
        node_to_ip[i] = config.get_endpoint(i);
    }
    local_node_id = getLocalNodeId();
}
```

After change: The constructor additionally instantiates `ConsistentHashRing` and `MembershipClient`. The `MembershipClient` connects to the configured coordinator and subscribes to topology updates. `count_of_node` and `node_to_ip` remain for endpoint lookup. `local_node_id` is still computed the same way. The private `getNodeId(int key)` method is removed.

13.2 KV Distributor::get

New logic:

```
primary_id = ring.getPrimaryNode(key)
try:
    if primary_id == local_node_id: return kv.Find(key)
    else: return kv_client.fetch(key, node_endpoints[primary_id])
catch (RPC exception):
    buddy_id = ring.getBuddyNode(key)
    if buddy_id == local_node_id: return kv.Find(key)
    else: return kv_client.fetch(key, node_endpoints[buddy_id])
```

13.3 KV Distributor::insert

New logic:

```
primary_id = ring.getPrimaryNode(key)
buddy_id   = ring.getBuddyNode(key)

// Write to primary; if unavailable, issue hinted write via buddy
try:
```

```

        if primary_id == local_node_id: kv.Insert(key, value)
        else: kv_client.insert(key, value, node_endpoints[primary_id])
    catch (RPC exception):
        hint_store.storeHint({primary_id, key, value, INSERT, now(),
primary_generation})
        // fall through to buddy write below (buddy becomes acting primary)

// Write to buddy
if buddy_id != primary_id:
    try:
        if buddy_id == local_node_id: kv.Insert(key, value)
        else: kv_client.insert(key, value, node_endpoints[buddy_id])
    catch: log [WARN]

```

13.4 KVDistributor::update and deleteKey

Same dual-write pattern as insert. Hinted handoff applies equally to `UPDATE` and `DELETE` operations.

13.5 KVDistributor::rebuildRing

New method called by `MembershipClient` on topology change:

```

void KVDistributor::rebuildRing(const std::vector<MemberRecord>&
live_members) {
    std::unordered_map<int, std::string> endpoints;
    for (const auto& m : live_members) {
        if (m.status == NodeStatus::ALIVE)
            endpoints[m.node_id] = m.endpoint;
    }
    ring_ = ConsistentHashRing(endpoints, virtual_nodes_per_node_);
}

```

This rebuild is fast (sub-millisecond for typical cluster sizes) and does not require any locks beyond a brief swap of the ring pointer.

13.6 Config Changes

```

// include/config.hpp
int          read_virtual_nodes_per_node() const;
std::string  read_membership_coordinator() const;
int          read_heartbeat_interval_ms() const;
int          read_suspect_threshold_ms() const;

```

```
int      read_dead_threshold_ms() const;
int      read_hint_expiry_ms()  const;
```

`kvm_server` does not need `ConsistentHashRing` or `MembershipClient` — routing is a client-side concern. If hinted handoff is implemented, `kvm_server` also gains a `HintStore` instance and the `hint_handoff` RPC handler so it can receive replayed hints from its buddy when it recovers.

14. Proposed Interface Changes

Note: The code snippets below are very minimal and may not be implemented, as it is, in the final implementation. It's here to provide a better clarity and visual on the changes being made on the existing codebase.

14.1 New Public Interface: `ConsistentHashRing`

```
// in include/ConsistentHashRing.hpp
// ...

class ConsistentHashRing {
public:
    ConsistentHashRing(
        const std::unordered_map<int, std::string>& node_endpoints,
        int virtual_nodes_per_node = 150
    );

    int getPrimaryNode(int key) const;
    int getBuddyNode(int key) const;
    int getNodeCount() const;

private:
    struct Token {
        uint32_t position;
        int      node_id;
    };
    std::vector<Token> ring_;
    int node_count_;
    int virtual_nodes_per_node_;

    static uint32_t hashKey(int key);
    static uint32_t hashToken(const std::string& token_str);
```



```
std::vector<Token>::const_iterator findToken(uint32_t position) const;
};
```

14.2 Modified KVDistriutor Interface

Public methods (Unchanged):

```
// Public (no changes visible to main_client.cpp)
int getNodeCount();
std::string get(int key);
void insert(int key, const std::string& value);
void update(int key, const std::string& value);
void deleteKey(int key);
```

The following private methods are added:

```
private:
    void rebuildRing(const std::vector<MemberRecord>& live_members);
    void issueHintedWrite(int intended_node_id, int key,
                          const std::string& value, HintOp op);
```

15. Failure Scenarios and Handling

15.1 Primary Node Down on Read

RPC exception from `KVClient::fetch`. `KVDistributor::get` catches it, logs a warning, and retries the read on the buddy node. The buddy holds a valid copy if buddy replication was in effect when the key was written.

15.2 Primary Node Down on Write

If hinted handoff is enabled, `KVDistributor::insert/update/deleteKey` catches the RPC exception, issues a hinted write (storing the `HintRecord` on the buddy's `HintStore`), and proceeds with the buddy write as the acting primary. Otherwise, the write fails over to the buddy without hint persistence. The client receives a success response only when the buddy path succeeds.

15.3 Buddy Node Down on Write

Primary write succeeds. Buddy write throws. Logged: `[WARN] Buddy write failed for key <k>`. Not retried. The key remains on the primary only until the buddy recovers. The buddy's

`HintStore` is not involved here. Hinted handoff is only triggered when the *primary* is unreachable.

15.4 Both Primary and Buddy Down on Read

Both RPC calls fail. `KVDistributor::get` returns an error string. No data loss is implied if persistent storage mode is in use. The data will remain on the downed nodes and will be accessible when they restart.

15.5 Primary Recovers After Hinted Writes

If hinted handoff is enabled, `MembershipService` broadcasts `NODE_RECOVERED`. The buddy's `MembershipClient` triggers hint delivery. The primary receives replayed writes in order and becomes authoritative again. The buddy deletes delivered hints.

15.6 Only One Physical Node Configured

`getBuddyNode` returns the same `node_id` as `getPrimaryNode`. The deduplication check prevents double-writing. The system functions as a non-replicated single-node store. If hinted handoff is enabled, it is a no-op because there is no separate buddy to hold hints.

15.7 Membership Service Coordinator Down

`MembershipClient` instances cannot receive topology updates. They continue operating with their last-known ring. New node additions/removals during the outage are not propagated. The system is available but cannot scale or respond to failures during this window. Recovery: when the coordinator restarts, it re-reads its durable membership log, reconstructs the membership list, and resumes broadcasting. This is the primary motivation for keeping the coordinator only as a fallback and, longer term, making the membership path highly available.

15.8 Hash Collision Between Two Tokens

Probability: For $V=150$ and $N=100$ nodes = 15,000 tokens, the birthday collision probability in a 2^{32} space is approximately $(15000^2) / (2 \times 2^{32}) \approx 0.026\%$. Negligible.

16. Industry Case Studies

This section examines how three production distributed key-value stores solve the same core problems: key routing, load balancing, fault tolerance, and membership management; and identifies the lessons most applicable to this project.

16.1 Redis Cluster: Hash Slots

Overview

Redis Cluster uses a deterministic, fixed-partition approach called **hash slots** rather than a continuous consistent hash ring. The entire key space is divided into exactly **16,384 slots**, numbered 0 to 16383.

Key Routing

Every key maps to a slot using a simple computation:

```
slot = CRC16(key) mod 16384
```

Redis uses CRC16 rather than a general-purpose hash like MurmurHash3. The choice of 16,384 slots was made deliberately: it fits in 2 KB as a bitfield (used in cluster gossip messages), and 16,384 is large enough to distribute smoothly across hundreds of nodes while remaining small enough to make slot-to-node mapping tables trivially cheap to store and communicate.

Hash tags are a notable feature: if a key contains a `{...}` pattern, only the substring inside the braces is hashed. For example, `{user:1001}.name` and `{user:1001}.email` both hash to the slot for `user:1001`, guaranteeing they land on the same node. This enables multi-key operations and transactions across related keys without cross-node coordination.

Cluster Topology and Slot Assignment

Slot-to-node assignment is maintained in a cluster-wide **slot map**, a 16,384-entry table mapping each slot to the node responsible for it. Every Redis node stores a copy of this map and gossips updates to peers. When a node joins, the cluster operator (or the `redis-cli --cluster rebalance` tool) assigns it a contiguous range of slots. When a node leaves, its slots are redistributed to other nodes.

```
Node A: slots 0      – 5460
Node B: slots 5461   – 10922
Node C: slots 10923  – 16383
```

Adding a node D might reassign:

```
Node A: slots 0      – 4095
Node D: slots 4096   – 5460  (migrated from A)
```

Only the keys in the migrated slots move. All other keys are undisturbed, this is functionally equivalent to what consistent hashing achieves, but implemented via explicit slot reassignment rather than a ring lookup.

Replication

Each primary node has one or more replica nodes (configured via `cluster-replicas` parameter, typically 1). Replication is asynchronous: writes go to the primary, which acknowledges the client immediately, and then replicates to replicas in the background. This means Redis Cluster is **AP** (available and partition-tolerant) under the CAP theorem: a primary failure may cause a small window of lost writes that were acknowledged but not yet replicated.

Automatic failover is handled by the cluster bus: replicas detect primary failure via gossip and the replica with the most up-to-date replication offset is promoted.

Membership and Failure Detection

Redis Cluster uses a **full-mesh gossip protocol** on the cluster bus (port 16379 by default, i.e., data port + 10000). Every node sends PING messages to a random subset of peers at regular intervals and expects PONG responses. A node that does not respond within `cluster-node-timeout` (default: 15 seconds) is marked `PFAIL` (possible failure). A node is marked `FAIL` when a quorum of nodes independently report it as `PFAIL`.

This is a decentralized approach, that is, there is no dedicated membership coordinator. The tradeoff is that convergence time for failure detection is bounded by gossip propagation delay rather than a single coordinator's timeout.

Comparison with This Project

Dimension	Redis Cluster	This Project
Routing mechanism	Fixed 16,384 hash slots	Continuous consistent hash ring
Load balancing	Explicit slot reassignment	Virtual nodes (automatic)
Replication	Async, configurable replica count	Synchronous buddy (1 replica)
Failure detection	Gossip (decentralized)	Gossip-based failure detection
Membership	Gossip, no coordinator	Gossip with coordinator fallback
Hash function	CRC16 mod 16384	MurmurHash3 (32-bit)
Hash tags	Yes (multi-key locality)	Not applicable (int keys)

16.2 Apache Cassandra: Consistent Hashing with Gossip

Overview

Cassandra uses consistent hashing on a 128-bit Murmur hash ring, with each node assigned one or more **tokens** (positions on the ring). Cassandra pioneered the use of virtual nodes (`num_tokens` , default 256 in recent versions) at production scale, providing automatic load balancing without manual token calculation.

Key Routing

```
partition_key_hash = murmur3_128(partition_key)
primary_node = first node clockwise from partition_key_hash
```

Cassandra's coordinator (the client-facing node that receives the query) uses a local copy of the token-to-node map (`TokenMetadata`) to route requests directly to the correct replica set — the same pattern this project's `KV DISTRIBUTOR` will use.

Replication

Cassandra's `SimpleStrategy` replicates each key to the next $N-1$ distinct nodes clockwise from the primary (where N is the `replication_factor` , typically 3). This is a direct generalization of this project's buddy replication (which uses $N=2$). Cassandra's `NetworkTopologyStrategy` extends this to place replicas across different datacenters.

Writes use a **quorum** model: a write is acknowledged when $\text{ceil}(N/2) + 1$ replicas confirm it. This provides strong consistency for reads that also use quorum ($R + W > N$). The buddy replication in this project is simpler ($N=2$, $W=1$ for writes, no quorum), which is appropriate given the scale.

Hinted Handoff in Cassandra

Cassandra implements hinted handoff almost identically to the design in Section 12 of this document. When a write cannot be delivered to a replica:

1. The coordinator (or one of the other replicas) stores a **hint**, a serialized version of the mutation tagged with the intended replica's endpoint.
2. Hints are stored in a dedicated system table (`system.hints`).
3. When the target node comes back online (detected via gossip), the hint holder replays all pending hints to it.
4. Hints expire after `max_hint_window_in_ms` (default: 3 hours in Cassandra 4.x) to prevent unbounded storage growth.

This is the direct precedent for the `HintStore` and hint expiry mechanisms described in Section 12.5.

Membership and Failure Detection

Cassandra uses the **Phi Accrual Failure Detector** rather than a fixed timeout. It maintains a sliding window of inter-arrival times for heartbeats from each peer and computes a `phi` value representing the probability that the peer has failed. A node is declared dead when `phi` exceeds a configurable threshold (typically 8). This produces more accurate failure detection under variable network conditions than a fixed `suspect_threshold_ms`.

For this project, the simpler fixed-timeout approach is appropriate. The Phi detector is noted here as a future improvement if false-positive failure detections become a problem.

Comparison with This Project

Dimension	Cassandra	This Project
Ring	128-bit Murmur	32-bit Murmur
Virtual nodes	Yes (num_tokens=256 default)	Yes (V=150 default)
Replication factor	N (configurable, typically 3)	N=2 (primary + buddy)
Write consistency	Quorum ($W > N/2$)	W=1 (primary write required)
Hinted handoff	Yes, system.hints table	Optional, if time permits
Failure detection	Phi Accrual	Fixed heartbeat timeout
Membership	Gossip (decentralized)	Gossip with coordinator fallback

16.3 Amazon DynamoDB

Overview

Amazon DynamoDB (original 2007 Dynamo paper) is one of the foundational references for consistent hashing in production. It uses a 128-bit MD5 ring with virtual nodes and introduced several techniques now standard in distributed KV stores.

Key Routing and Preference Lists

DynamoDB assigns each key to a **preference list** of N nodes (N=3 typically), where the first node on the list is the coordinator for that key. Unlike this project's binary primary/buddy model,

Dynamo allows any of the N nodes to serve reads and writes, using a **sloppy quorum** (reads and writes succeed when R or W nodes out of N respond, where $R + W > N$).

Hinted Handoff in Dynamo

Dynamo's hinted handoff is nearly identical to the mechanism in Section 12. During network partitions or node failures, writes that cannot reach one of the N preferred nodes are accepted by the first available node outside the preference list. The hint (containing the intended recipient's identity) is stored alongside the write. Once the intended node recovers, the hint holder forwards the writes and deletes the local hint copy.

The Dynamo paper explicitly notes that hint storage must be bounded: nodes that accumulate too many hints are deprioritized for future hinted writes to prevent a healthy node from becoming a bottleneck.

Membership

Dynamo uses a hybrid approach: a **ring membership service** handles explicit joins/leaves, while gossip propagates the ring state to all nodes. New nodes are assigned tokens manually by an operator to ensure balanced placement. This predates widespread adoption of virtual nodes.

Comparison with This Project

Dimension	DynamoDB (Dynamo)	This Project
Ring	128-bit MD5	32-bit MurmurHash3
Replication	$N=3$, sloppy quorum	$N=2$, primary mandatory
Hinted handoff	Optional, if time permits	Optional, if time permits
Membership	Ring service + gossip	Gossip with coordinator fallback
Conflict resolution	Vector clocks	Last-write-wins (implicit)

16.4 Summary

The following table maps each design decision in this project to the precedent set by industry systems:

Design Decision	This Project's Choice	Industry Precedent
Ring type	Continuous consistent hash (32-bit)	Cassandra (128-bit), Dynamo (128-bit)
Slot-based alternative	Not used	Redis (16,384 slots)
Virtual nodes	Yes, V=150	Cassandra (num_tokens=256), Dynamo
Hash function	MurmurHash3 (32-bit)	Cassandra (MurmurHash3 128-bit)
Replication	Primary + 1 buddy	Redis (1 replica), Dynamo (N=3)
Write path	Primary required, buddy best-effort	Redis (async replica), DynamoDB (quorum)
Hinted handoff	Optional, if time permits	Cassandra, DynamoDB
Membership	Gossip with coordinator fallback	DynamoDB (hybrid), Redis (gossip)
Failure detection	Fixed heartbeat timeout	Redis (gossip + timeout), Cassandra (Phi)

17. Future Scope / Improvements

- **Hinted handoff implementation if time permits.** This is the main stretch feature for the first pass; if it slips, the design still works with buddy replication plus read fallback.
 - **Re-replication after failure beyond hinted handoff.** When hints expire or a node is permanently lost, keys with only one live copy need to be re-replicated to a new buddy. This requires a background repair process and is a separate work item.
 - **Client-side ring consistency across multiple client processes without the membership service.** Once dynamic node addition/removal is active, all clients must have the membership service running to maintain a consistent ring view.
 - **Phi Accrual Failure Detector.** The fixed-timeout heartbeat approach is used for now; the Phi detector is noted as a future improvement.
 - **Consensus/leader election for membership coordinator HA.** The coordinator node can be a single point of failure if gossip protocol is not implemented.
 - **Multi-key transactions.** Redis hash tags enable multi-key locality; this project uses integer keys and does not require transaction support.
-